Sintaxis computacional

Borja Navarro Colorado
Procesamiento del Lenguaje Natural I
Grado en IA - Universidad de Alicante

5 de noviembre de 2026

Índice

En esta sesión veremos:

- Objeto de la sintaxis (justificación).
- Modelos formales basado en constituyentes.
- Modelos formales basado en dependencias.

Fundamentos de sintaxis (repaso)

- **Objeto** de la sintaxis:
  - Organización de las palabras en el discurso.
  - Orden y combinación de palabras dentro de la oración.
  - Relaciones jerárquicas (árboles) entre las palabras de una oración.
- Estas agrupaciones y relaciones determinan el significado final del texto (principio de composicionalidad).

Relevancia de la sintaxis

No es lo mismo:

- *vivir en la calle del medio* que *vivir en el medio de la calle*.
- *me río en el baño* que *me baño en el río*.

Constituyentes vs. dependencias

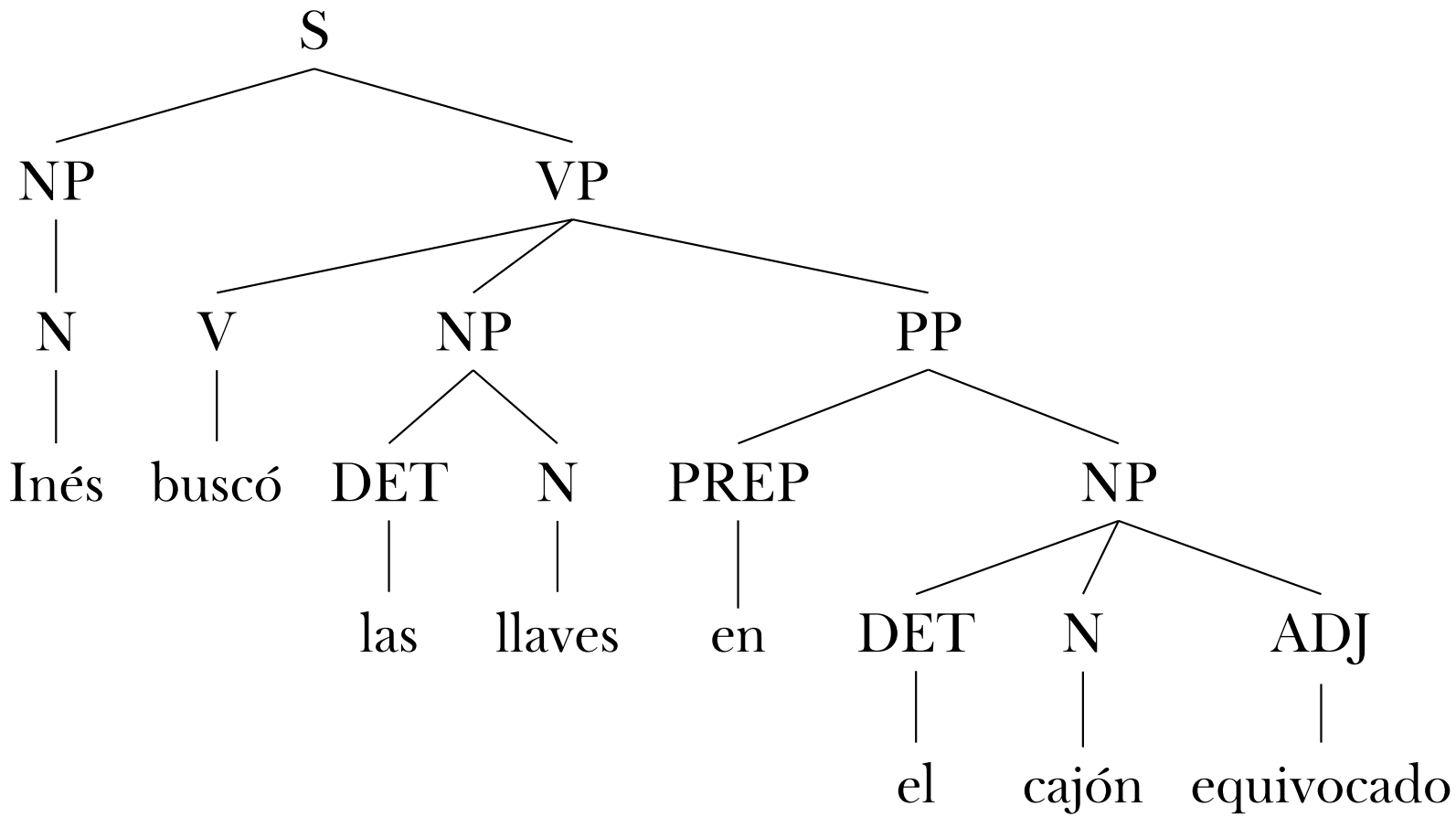


Figura: Análisis de constituyentes.

Constituyentes vs. dependencias

Inés buscó las llaves en el cajón equivocado.

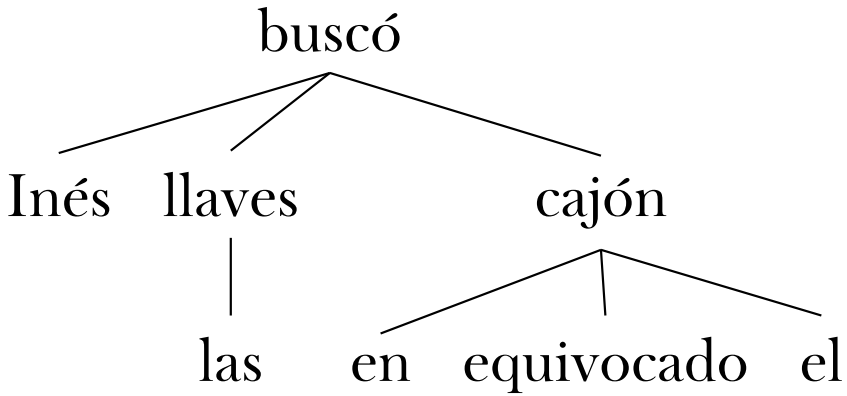


Figura: Análisis de dependencias.

Modelos basados en constituyentes

Análisis de constituyentes

- **Constituyente**: conjunto de palabras (una o más) que pueden actuar en una oración como una unidad.
- Analizar estas secuencias:
  - *El hombre vestido de azul pinta la escalera*
  - (*)*El azul de vestido pinta hombre escalera la de*
  - (?)*El hombre vestido pinta la escalera de azul*
  - *El hombre pinta la escalera vestido de azul*

Análisis de constituyentes

- Sintagma (*phrase*): expansión de un núcleo.
- El tipo de sintagma lo determina la categoría gramatical que actúa de núcleo: nominal, verbal, adjetivo, preposicional, adverbial...

Gramáticas formales

- La información sintáctica se representa mediante *GRAMÁTICAS FORMALES*
- Las gramáticas formales especifican mediante **reglas** las agrupaciones sintagmáticas correctas ("gramaticales") frente a las incorrectas ("agramaticales")

  Cómo se agrupan las palabras en sintagmas y tipos de sintagmas.
- El tipo básico de gramática formal son las **Gramáticas Independientes de Contexto** o *Context Free Grammars* (CFG).

Context Free Grammars

Una CFG está formada por:

- Un conjunto de símbolos no terminales (NT)
- Un conjunto de símbolos terminales (T)
- Un conjunto de reglas de reescritura o producción (P)

Context Free Grammars

Pepe come una manzana

$$
\begin{aligned}
T &= \{Pepe, manzana, come, una\},\\
NT &= \{S, NP, VP, nprop, n, v, det\},\\
\text{P:}&\\
&S \rightarrow NP \quad VP\\
&NP \rightarrow nprop\\
&NP \rightarrow det\ n\\
&VP \rightarrow v\\
&VP \rightarrow v\ NP\\
&nprop \rightarrow \text{“}Pepe\text{”}\\
&n \rightarrow \text{“}manzana\text{”}\\
&det \rightarrow \text{“}una\text{”}\\
&v \rightarrow \text{“}come\text{”}
\end{aligned}
$$

Cuadro: Gramática independiente de contexto o *context free grammar* (CFG)

De esta CFG, se deriva el siguiente el árbol (3):

Análisis de constituyentes

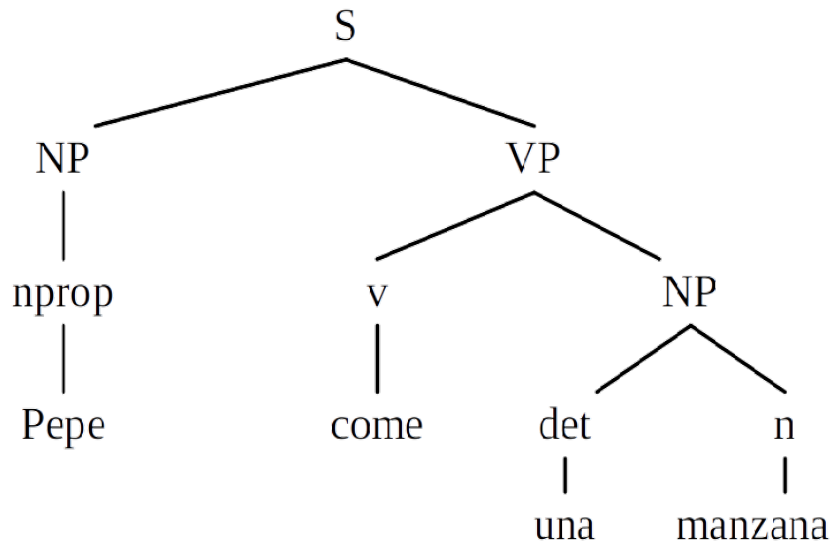


Figura: Ejemplo de análisis sintáctico de constituyentes

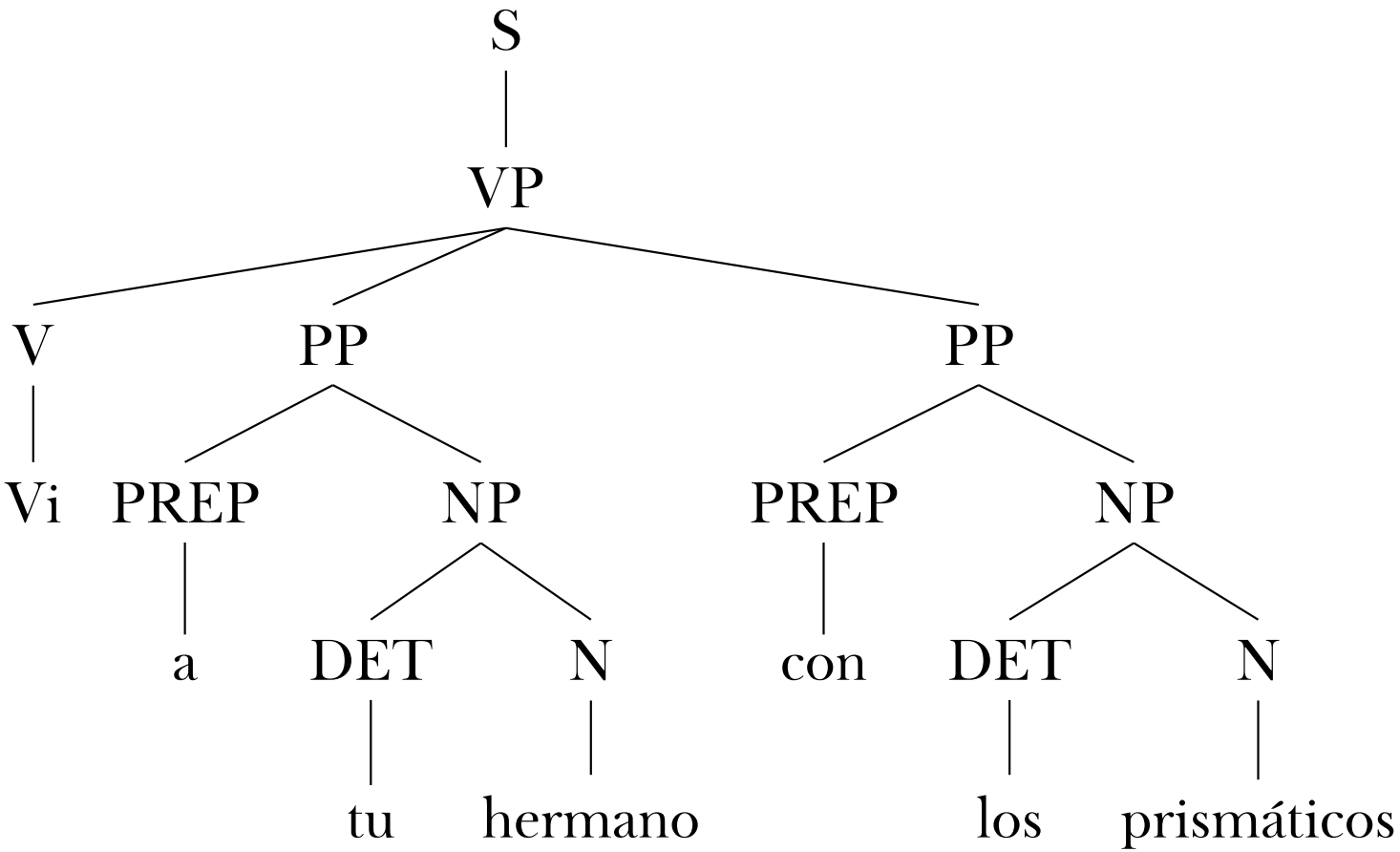
Proceso de análisis

Visualización del proceso de análisis con la herramienta NLTK:

- Instala NLTK: https://www.nltk.org/install.html
- Para un análisis recursivo descendiente ejecuta en Python:
  - `import nltk`
  - `nltk.app.rdparser()`
- Para un análisis ascendente tipo "desplaza y reduce" (*shift-reduce*) ejecuta:
  - `nltk.app.srparser()`

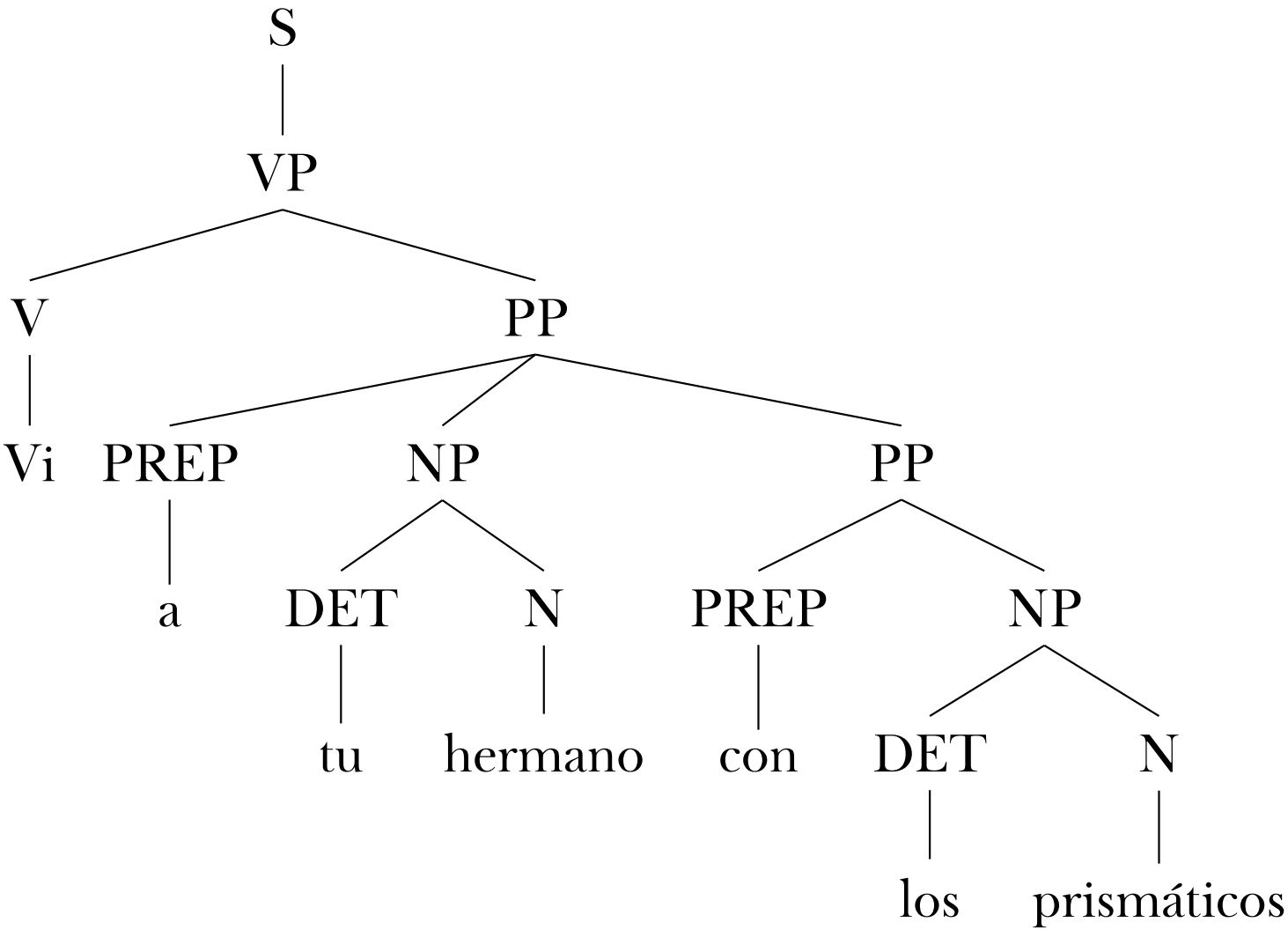
Problema de ambigüedad

Vi a tu hermano con los prismáticos



Problema de ambigüedad

Vi a tu hermano con los prismáticos



TreeBanks

- Corpus en el que cada oración ha sido analizada, haciendo explícitos todos los árboles sintáticos.
  - Cada árbol revisado y corregido por un lingüista experto.
- Necesarios para generar gramáticas CFG a partir de textos reales.
  - Todos los árboles de un TreeBank constituyen ya una gramática.
  - Ej. *Probabilistic Context Free Grammars* (PCFG).

TreeBanks

Ejemplos de treebanks:

- PennTreeBank: https://catalog.ldc.upenn.edu/LDC99T42
- Ancora (antiguo Cast3LB): https://clic.ub.edu/corpus/es/ancora
- Y muchos más: https://en.wikipedia.org/wiki/Treebank

PennTreeBank

```
>>> from nltk.corpus import treebank
>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> print(t)
(S
  (NP-SBJ
    (NP (NNP Pierre) (NNP Vinken))
    (, ,)
    (ADJP (NP (CD 61) (NNS years)) (JJ old))
    (, ,))
  (VP
    (MD will)
    (VP
      (VB join)
      (NP (DT the) (NN board))
      (PP-CLR
        (IN as)
        (NP (DT a) (JJ nonexecutive) (NN director)))
      (NP-TMP (NNP Nov.) (CD 29))))
  (. .))
```

Figura: Ejemplo corpus PennTreeBank desde NLTK

PennTreeBank tag set:

- https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html
- https://gist.github.com/nlothian/9240750

Ancora corpus

```
>>> import nltk
>>> from nltk.corpus import cess_esp as cess
>>> oracion  = cess.parsed_sents()[0]
>>> print(oracion)
(S
  (sn-SUJ
    (espec.ms (da0ms0 El))
    (grup.nom.ms
      (ncms000 grupo)
      (s.a.ms (grup.a.ms (aq0cs0 estatal)))
      (sn
        (grup.nom.ms
          (np00000 Electricité_de_France)
          (sn (grup.nom.ms (Fpa -Fpa-) (np00000 EDF) (Fpt -Fpt-)))))))
  (grup.verb (vmis3s0 anunció))
  (sadv-CCT
    (grup.adv (rg hoy) (sn (Fc ,) (grup.nom.ms (W jueves)) (Fc ,))))
  (sn-CD
    (espec.fs (da0fs0 la))
    (grup.nom.fs
      (ncfs000 compra)
      (sp
        (prep (spcms del))
        (sn
          (grup.nom.ms
            (Zp 51_por_ciento)
            (sp
              (prep (sps00 de))
              (sn
```

Figura: Ejemplo corpus Ancora-ESP desde NLTK

- https://clic.ub.edu/corpus/es/ancora-descarregues
- https://huggingface.co/datasets/CLiC-UB/AnCora-ES

Ampliación de las CFG

- Las CFG son muy simples para describir las estructuras complejas de un idioma.

  Ejemplo: una regla tipo `NP -> Det, N` daría como gramatical una oración del tipo (*) *La coche*.
- Ampliación mediante estructuras de rasgos.

Estructura o matrices de rasgos

CFG ampliadas:

- `NP(NUM, GEN) --> D(NUM, GEN), N(NUM, GEN)`
- `NP(sing, masc) --> D(sing, masc), N(sing, masc)`
- Reglas de unificación

Estructura o matrices de rasgos

CFG ampliadas:

- `NP(NUM, GEN) --> D(NUM, GEN), N(NUM, GEN)`
- `NP(sing, masc) --> D(sing, masc), N(sing, masc)`
- Reglas de unificación

Matrices de rasgos:

- Permiten la representación formal de información lingüística.
- Formado por pares atributo-valor.
- Se pueden anidar unas estructuras en otras.
- Admite listas (entre corchetes) y lista vacía.
- Permite aplicar unificación.

Modelos lingüísticos formales con estructuras de rasgos tipificadas:

- HPSG *Head-driven Phrase Structure Grammars*: https://langsci-press.org/catalog/book/478
- LFG *Lexical-Functional Grammars*: https://ling.sprachwiss.uni-konstanz.de/pages/home/lfg/, https://langsci-press.org/catalog/book/312
- CCG *Combinatory Categorial Grammar*: https://groups.inf.ed.ac.uk/ccg/
- etc.

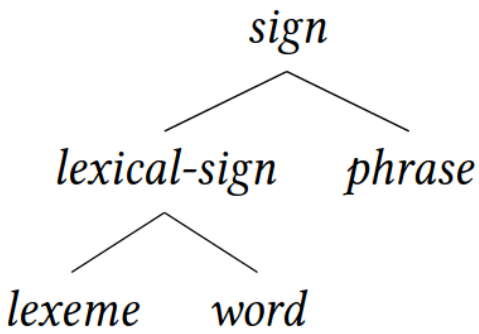
En el léxico, cada palabra es una matriz de rasgos con todos sus datos gramaticales (género, número, subcategorización, etc.)

HPSG

Análisis de un caso: la *Head-driven Phrase Structure Grammar*

- Amplía la representatividad de las CFG con estructuras de rasgos.
- Una gramática HPSG está formada por tres elementos:
  - Tipos (*types*)
  - Rasgos (*features*)
  - Restricciones (*constraints*)

Types:



- El conjunto de *lexical-signs* y sus restricciones constituye el **lexicón** de un idioma.
- El conjunto de *phrases* y sus restricciones conforman la **sintaxis** de un idioma.

lexical-signs y *phrases* se representan mediante estructuras de rasgos.

$$\begin{bmatrix} \mathit{WORD} & \\ \text{PHON} & \langle \mathit{CAT} \rangle \\ \text{SYNSEM} & \begin{bmatrix} \text{POS} & n \\ \text{NUM} & sg \end{bmatrix} \end{bmatrix}$$

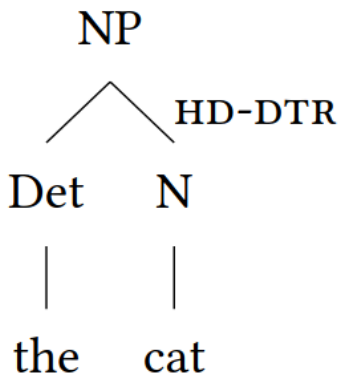
Figura: Estructura de rasgos de un nombre

lexical-signs y *phrases* se representan mediante estructuras de rasgos.

$$\begin{bmatrix} \textit{phrase} \\ \textsc{phon} \quad \langle \textit{the, cat} \rangle \\ \textsc{synsem} \ \text{NP} \\ \textsc{dtrs} \quad \left\langle \begin{bmatrix} \textsc{phon} \quad \langle \textit{the} \rangle \\ \textsc{synsem} \ \text{Det} \end{bmatrix}, \boxed{1} \begin{bmatrix} \textsc{phon} \quad \langle \textit{cat} \rangle \\ \textsc{synsem} \ \text{N} \end{bmatrix} \right\rangle \\ \textsc{hd-dtr} \ \boxed{1} \end{bmatrix}$$

- DTRS: *daughters*
- HD-DTR: *head daughter*

La matriz de rasgos anterior representa la misma información que este árbol CFG:



Pero la matriz de ragos permite incluir muchísima información de cada palabra o sintagma:

- categoría gramaticales
- estructuras de subcategorización
- contenido semántico (*feat. content*)
- deixis y contenido pragmático (*feat. context*)
- ...

- Las *constraints*, finalmente, imponen restricciones a modo de reglas.
- Ejemplo: la unificación.

  Dos estructuras de rasgos tipo *word* podrán formar una estructura tipo *phrase* si algunos de sus rasgos son compatibles, como el género o el número de determinantes y nombres.

HPSG

Implementaciones HPSG:

- https://github.com/delph-in
- https://pypi.org/project/PyDelphin/

Actividad final

Analiza con la demo de Freeling las siguientes oraciones y localiza los errores.

- `Veinte años hace, poco más o menos, que al declinar una tarde del mes de junio un joven de hermosa presencia atravesaba a caballo los campos pintorescos que riega el Tínima.`
- `Un joven de hermosa presencia atravesaba a caballo los campos pintorescos que riega el Tínima al declinar una tarde del mes de junio hace veinte años poco más o menos.`
- `El sol terrible de la zona tórrida se acercaba a su ocaso entre ondeantes nubes de púrpura y de plata.`

- Demo: https://nlp.lsi.upc.edu/freeling/
- TagSet: https://freeling-user-manual.readthedocs.io/en/v4.2/tagsets/

Modelos basados en dependencias

Análisis de dependencias

- *Dependency Grammar*
- Es el formalismo de representación sintáctica más utilizado en PLN en la actualidad.
  - SpaCy: https://spacy.io/
  - Stanza: https://stanfordnlp.github.io/stanza/
  - La mayoría de las propuestas de análisis sintáctico computacional.

Gramáticas de dependencias

- Representan de manera formal las dependencias sintácticas entre las palabras de una oración.
- Se especifica directamente:
  - qué palabras dependen sintácticamente de qué otra palabras y
  - qué tipo de relación o dependencia mantienen entre sí (Figura 7).

Análisis de dependencias

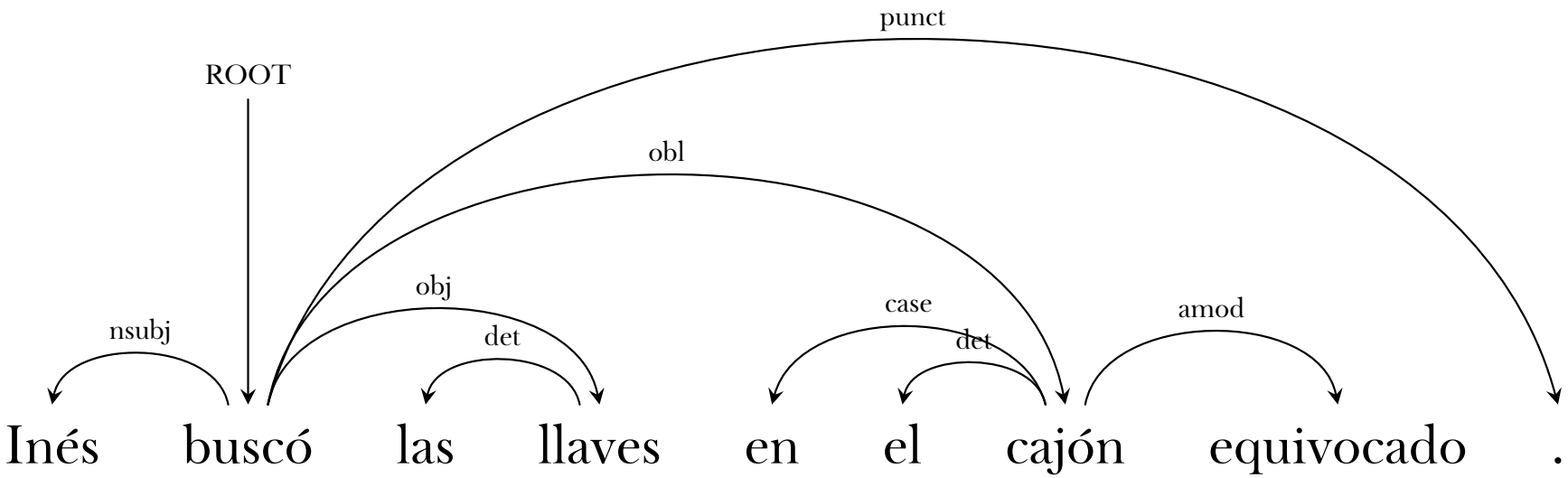


Figura: Ejemplo completo de análisis de dependencias.

Análisis de dependencias

- Formalmente, la relación sintáctica entre dos palabras es un **arco binario directo**.
  - elemento principal: **núcleo** o *head*.
  - elemento relacionado o dependiente: **complemento** o *dependent*.
- **Estructura de dependencia tipificada** (*typed dependency structure*): cada arco está categorizado con el tipo de dependencia sintáctica.
  - Los tipos de arcos están predefinidas.
- En la gramática tradicional del español a estas categorías se las denomina **funciones sintácticas**.

Análisis de dependencias

Observar en el ejemplo anterior (Fig. 7):

- Los arcos son dirigidos: van siempre de una palabra núcleo a una palabra objetivo (la palabra dependiente). No son, por tanto, relaciones simétricas.
- El árbol se inicia con una palabra principal, categorizada como *root* (en este caso, "busco").
- El resto de palabras quedan relacionadas dos a dos según su dependencia.
- La tipología de de dependencias está basada, en este caso, en el modelo de dependencias universales (*Universal Dependencies*, ver 43):

Análisis de dependencias

Jurafsky y Martin (2026) indican algunas ventajas del análisis de dependencias:

1. Es más fácil extraer las relaciones entre las palabras, ya que están codificadas directamente.
2. El modelo de **constituyentes**, dado que hace agrupaciones, **es bastante dependiente de la posición de las palabras** en la oración, mientras que el de dependencias no. Especialmente útil para lenguas con un orden de palabras relativamente libre y de morfología rica como el vasco.
3. Las **relaciones sintácticas de dependencias se asemejan a las relaciones semánticas** entre los constituyentes, por lo que es un formalismo más apropiado para un posterior análisis semántico como el análisis de roles semánticos o eventos.

Análisis de dependencias

Las dependencias sintácticas tienen una larga tradición en lingüística teórica:

- Tesnière (1959) *Élément de syntaxe structurale* desarrolla el concepto de sintaxis de dependencia.
- *Meaning-Text Theory* de Mel'cuk (1988) desarrolla un modelo formal de dependencias, al igual que
- *Word Grammar* de Hudson (1984)
- *Functional Generative Description* (FDG) de Sgall et al. (1986)
- y otras...

Formato de representación

Formato de representación estándar: CoNLL (*Conference on Natural Language Learning*) y CoNLLL-U.

ID	Token	Lema	Cat_gram	Depend	Tipo_depend.
1	Los	el	DET	2	det
2	hombres	hombre	NOUN	6	nsubj
3	que	que	PRON	4	nsubj
4	fuman	fumar	VERB	2	acl
5	puro	puro	NOUN	4	obj
6	tienen	tener	VERB	0	root
7	cara	cara	NOUN	6	obj
8	de	de	ADP	9	case
9	canguro	canguro	NOUN	7	nmod
10	.	.	PUNCT	6	punct

Formato de representación

- Cada palabra tiene un identificador único (primera columna).
- La información sintáctica de dependencias está codificada en las columnas 5 (Depend) y 6 (Tipo_depend).
- La columna 5 indica el identificador de la palabra con la que tienen algún tipo de dependencia, y la columna 6 el tipo de dependencia.

Dependencias Universales

- Estándar *de facto* en PLN para representar relaciones de dependencias.
- **Objetivo**: un modelo de representación gramatical formal común a diversos idiomas ("universal").
  - Incluye categorías gramaticales, análisis morfológico y análisis sintáctico de dependencias.
- Idiomas incluidos. Ver https://universaldependencies.org/

Dependencias universales

Tres tipos de dependencias:

1. **Argumentales** (*core arguments*), exigidas por la semántica del verbo (un complemento directo, por ejemplo).
2. **No argumentales** (*non-core dependents*), completan la oración pero que sin ella ésta tendría sentido, como los complementos circunstanciales.
3. **Nominales**, dependencias que se establecen con un nombre (complemento nombre-adjetivo).

https://universaldependencies.org/u/dep/

Análisis de dependencias

A su vez, el constituyente dependiente puede ser:

- nominal (sintagma nominal en sentido estricto o preposicional),
- cláusula (subordinadas),
- modificador (adverbios, adjetivos...),
- palabra funcional (determinantes, preposiciones (*case*...)).

Análisis de dependencias

Principales etiquetas:

	Nominals	Clauses	Modifier words	Function Words
Core arguments	nsubj obj iobj	csubj ccomp xcomp		
Non-core dependents	obl vocative expl dislocated	advcl	advmod* discourse	aux cop mark
Nominal dependents	nmod appos nummod	acl	amod	det clf case
Coordination	**Headless**	**Loose**	**Special**	**Other**
conj cc	fixed flat	list parataxis	compound orphan goeswith reparandum	punct root dep

* The advmod relation is used for modifiers not only of predicates but also of other modifier words.

Figura: https://universaldependencies.org/u/dep/

Análisis de dependencias

Esta configuración general se adapta a cada idioma:

https://universaldependencies.org/

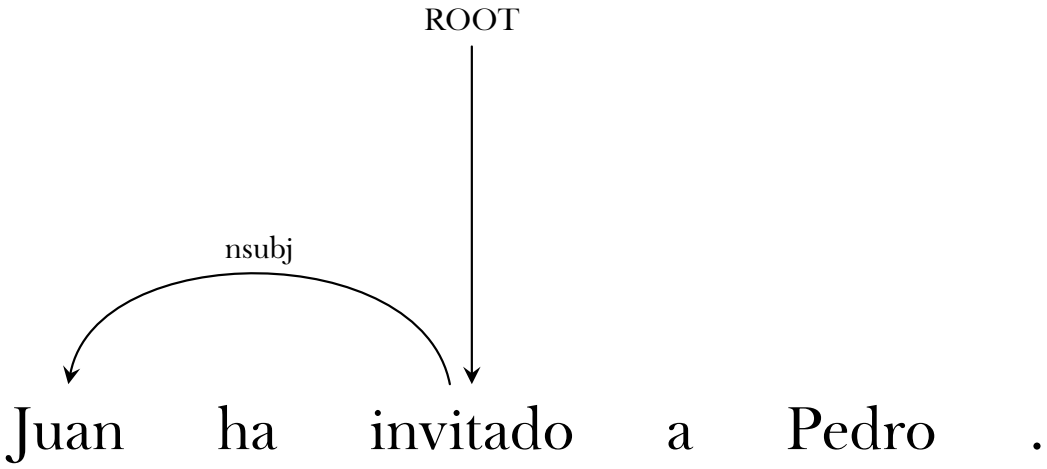
- Arabic: https://universaldependencies.org/ar/index.html
- Spanish: https://universaldependencies.org/es/index.html
- etc.

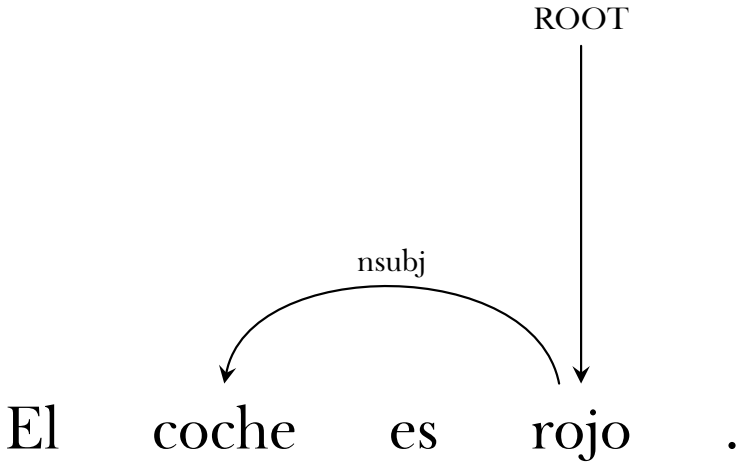
Dependencias

`nsubj`: nominal subject

- Sujeto sintáctico o proto-agente de una cláusula.
- Núcleo: nombre o pronombre (en caso de elipsis, otra forma como el adjetivo).
- Suele ser dependiente de un verbo, pero no siempre (ej. verbos copulativos la palabra dependiente es el atributo (nombre o adjetivo)).
- Variantes:
  - `nsubj:pass` sujeto oraciones pasivas
- https://universaldependencies.org/u/dep/nsubj.html

Análisis de dependencias



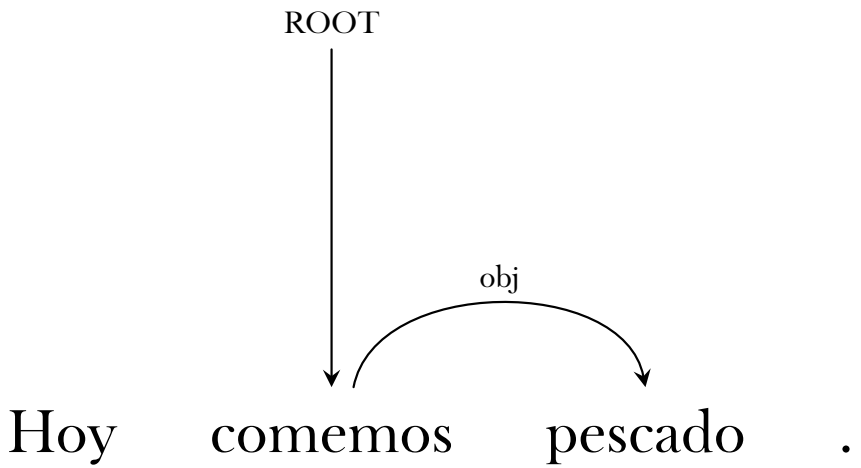


Dependencias

`obj`: object

- Proto-paciente: sintagma nominal que denota la entidad sobre la que se actúa o que experimenta un cambio de estado o movimiento.
- Caso acusativo (en idiomas con casos)
- Solo puede haber uno por oración. Si pudieran ser varios, será `obj` el que mayor relación o dependencia tenga con el verbo (*paciente*), el resto serán `iobj`.
- https://universaldependencies.org/u/dep/obj.html

Análisis de dependencias

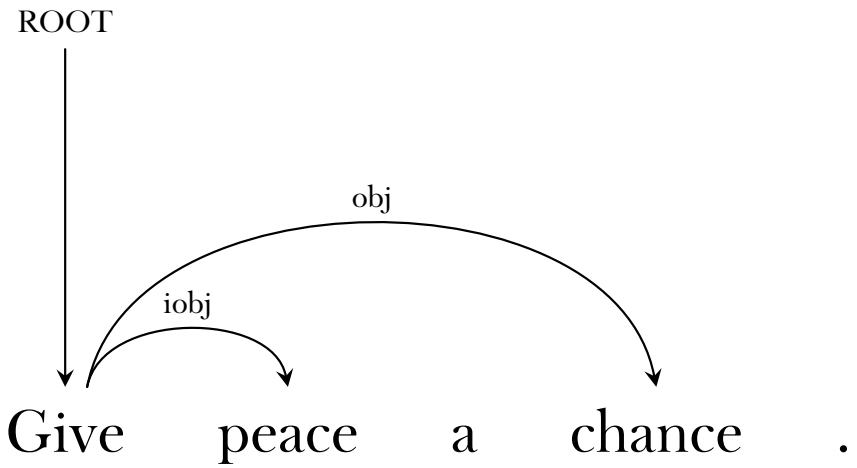


Dependencias

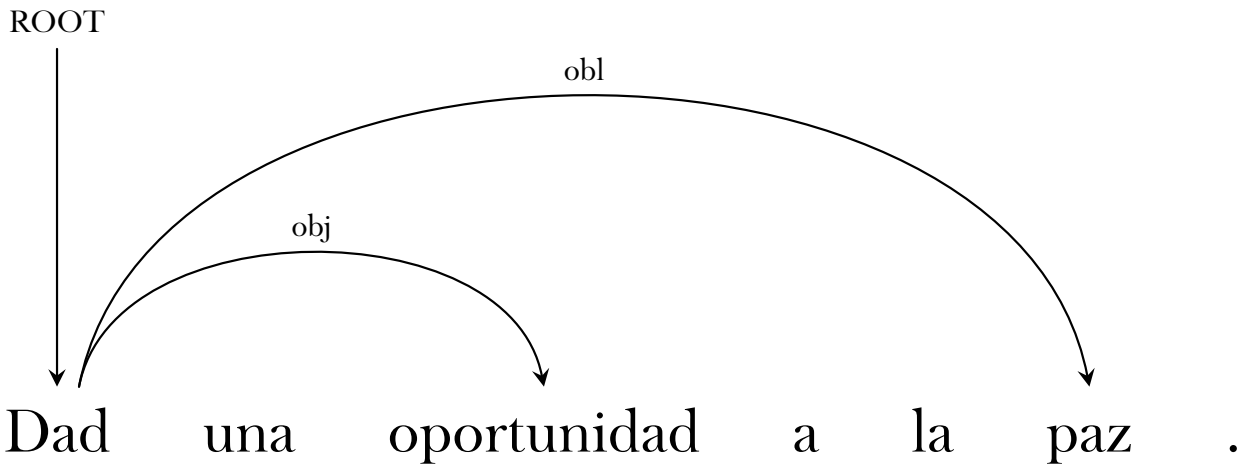
`iobj`: indirect object

- Sintagma nominal argumental (*core argument*) pero que no se puede marcar como `nsubj` ni como `obj` (verbos ditransitivos).
- Caso dativo
- No es el complemento indirecto de la gramática española (ver luego `obl:arg`) porque debe ser sintagma nominal.

Análisis de dependencias



pero

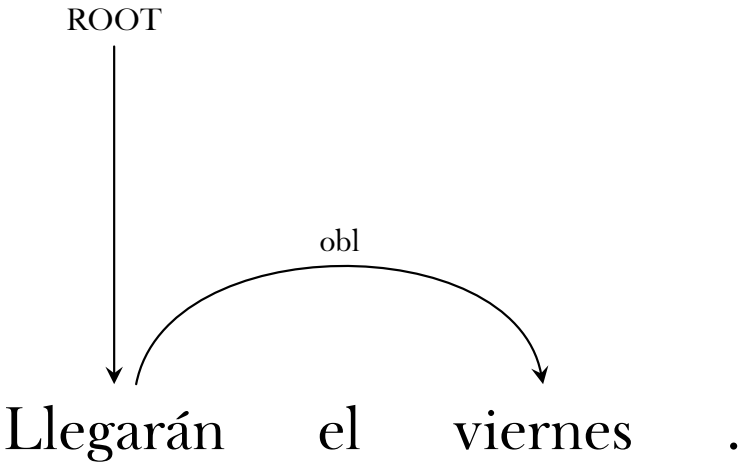


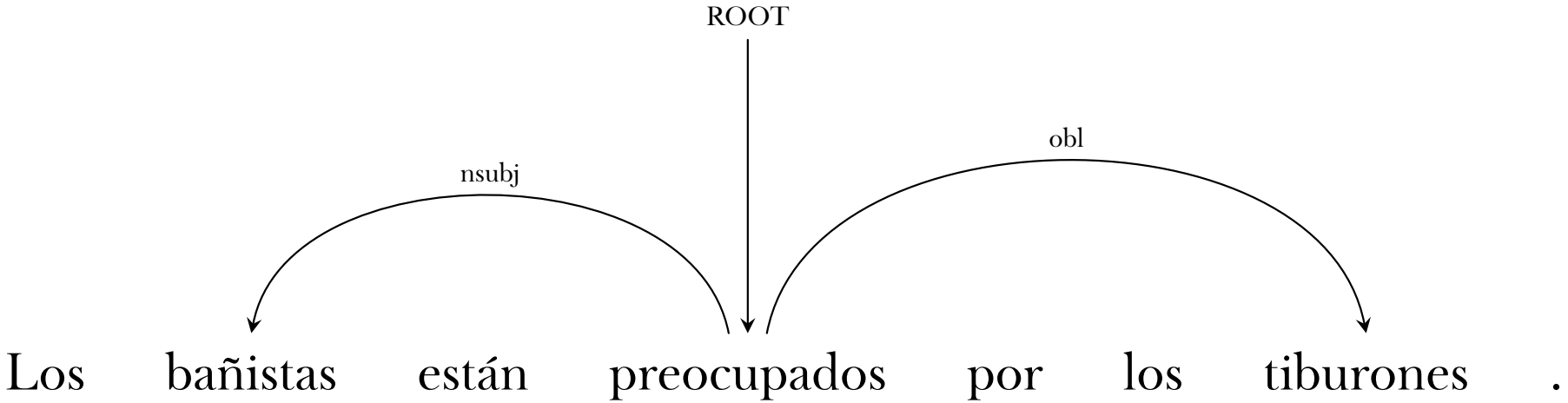
Dependencias

`obl`: oblique nominal

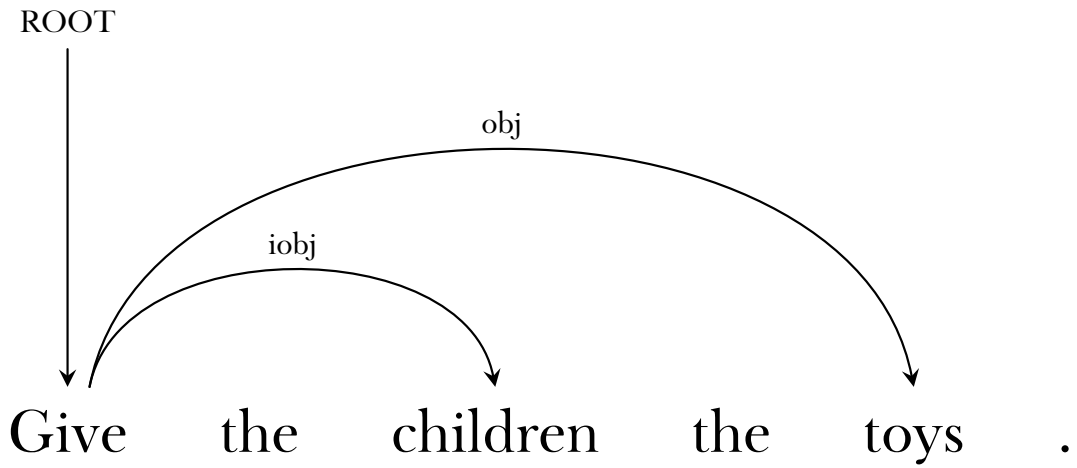
- Relaciones indirectas (*non-core, oblique*)
- Nominal que depende de una expresión verbal, adjetiva o adverbial.
- https://universaldependencies.org/u/dep/obl.html

Análisis de dependencias

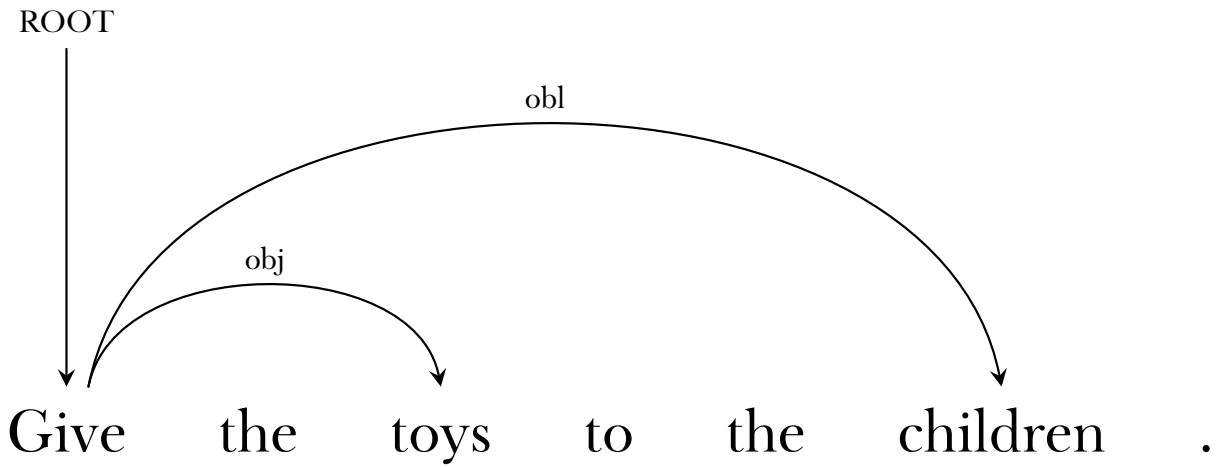




Análisis de dependencias



pero



Dependencias

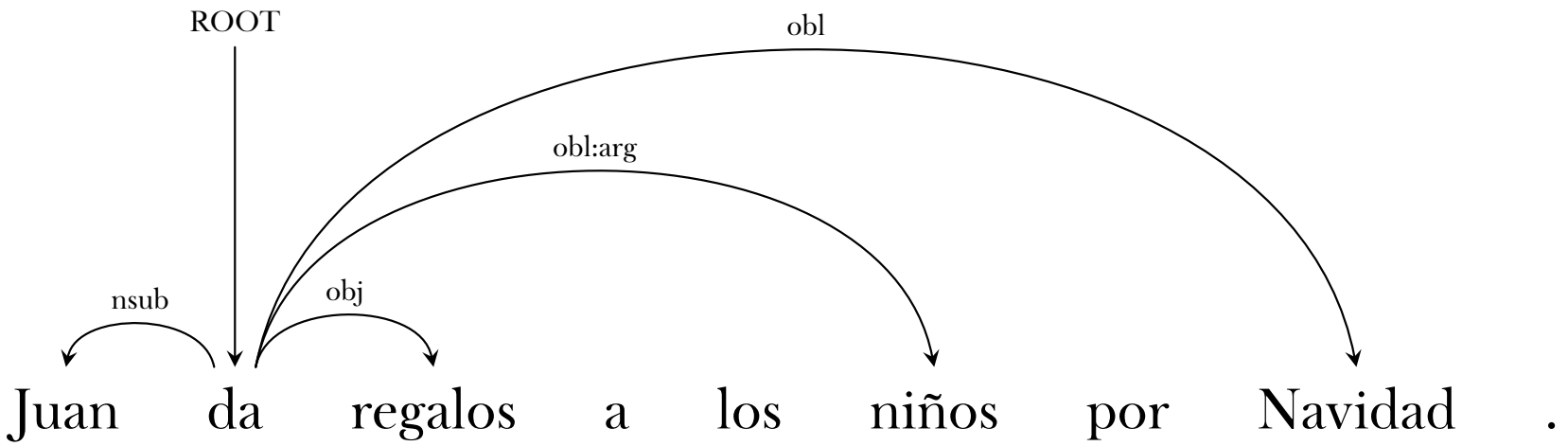
Dos casos especiales de `obl`:

- `obl:agent`: complemento agente de oración pasiva (Vid. `nsub:pass`).

  https://universaldependencies.org/en/dep/obl-agent.html
- `obl:arg`: *oblique argument*, complemento indirecto argumental.

  https://universaldependencies.org/u/dep/obl-arg.html

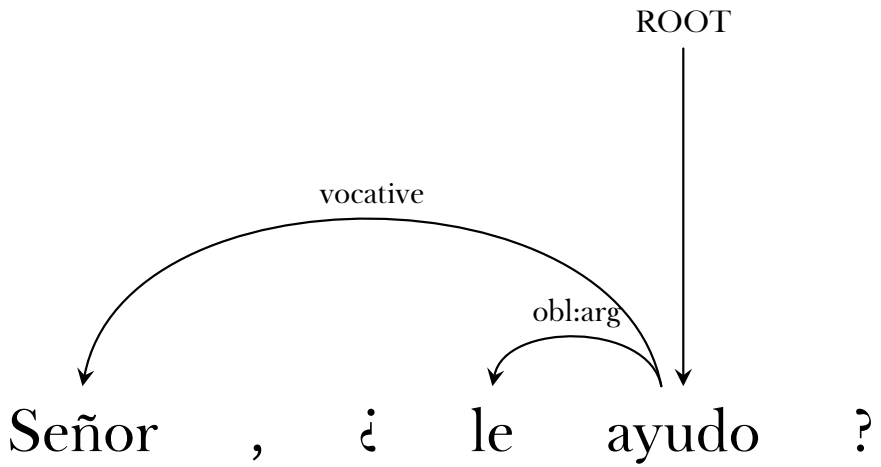
Análisis de dependencias



Dependencias

`vocative`:

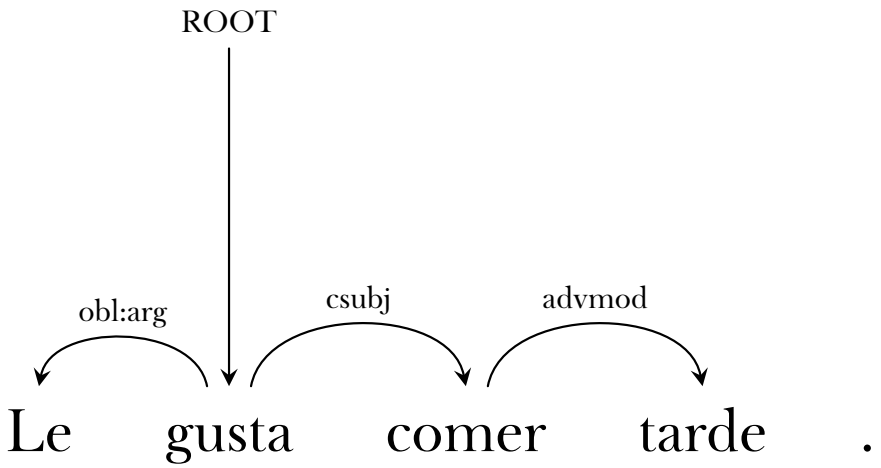
- La persona a la que se dirige un diálogo



Dependencias

`csubj`: clausal subject.

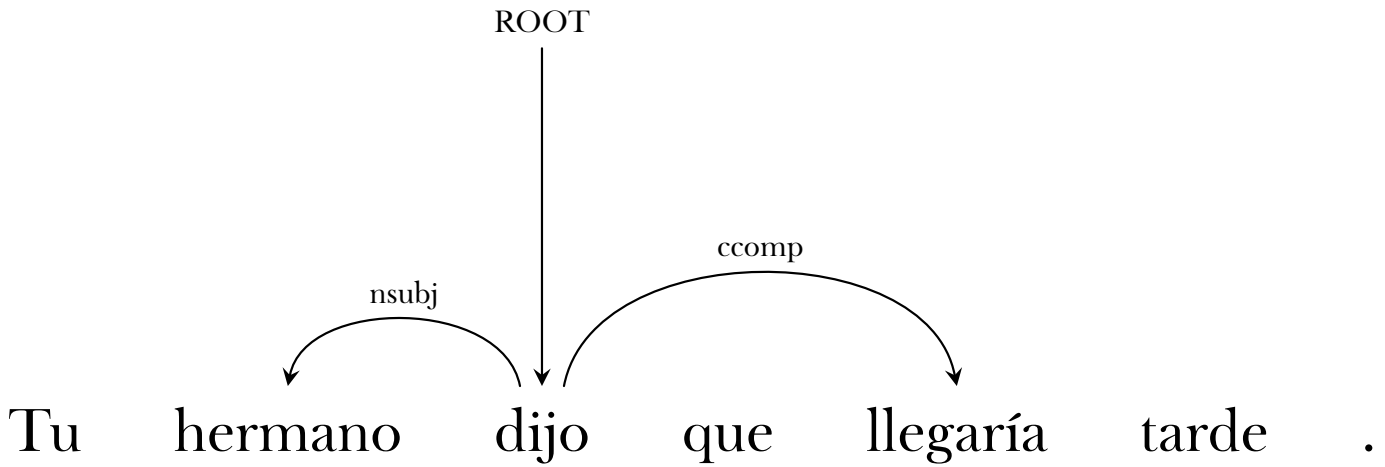
- Sujeto oracional (no nominal)



Dependencias

`ccomp`: clausal complement

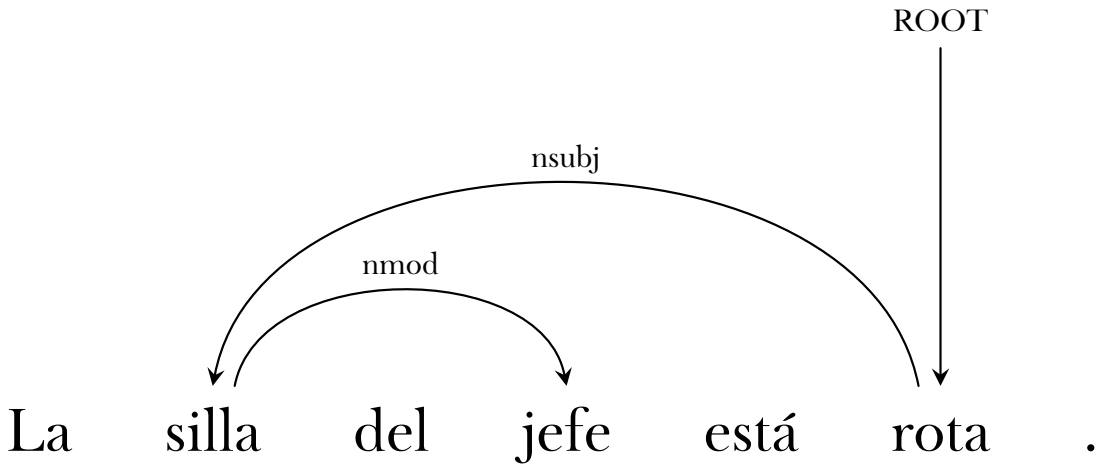
- Objeto oracional (oración subordinada sustantiva de objeto directo) o adjetivo.



Dependencias

`nmod`: nominal modifier

- Complemento del nombre.



Proceso de análisis

- Análisis de dependencias basado en transiciones (Nivre 2014)
- Modelos neuronales
- Modelo Tranformer Biafin.

Proceso de análisis

Transition-based dependency parsing de Nivre (2014) es el sistema de análisis de dependencias estándar.

- Análisis ascendente *shift-reduce*
- Aprende la mejor acción (*oracle*) en cada caso (SHIFT, LEFTARC, RIGHTARC) a partir de TreeBank (SVM, etc.)
- Mejoras:
  - *Arc eager* (arco impaciente): relaciona palabra con su núcleo lo antes posible, mantiene la palabra nueva en lista (fuera de la pila) y, si aplica RIGHTARC, la introduce en la pila.
  - *Beam search* (búsqueda en haz o abanico): se calculan en paralelo las posibles transiciones y se mantienen las k más probables en una "agenda".

Proceso de análisis

Modelos basado en grafos o *graph-based*.

- Aplican teoría de grafos. Apropiado para oraciones largas. Permite generar árboles no-proyectivos (cruce de ramas).
- Asignación de pesos a cada arco. Entrenamiento con *tree bank* y conjunto de *features*.
- Selecciona el árbol de expansión máxima (valor más alto).

Proceso de análisis

Biaffine Attention Mechanism (Dozat y Manning, 2017): método neuronal para asignar pesos a los arcos de dependencias.

- Un *encoder* (BiLSTM, *Transformer*) genera un vector contextual r_i para cada token i de cada oración.
- Mediante redes FFN, el vector se desacopla en vectores específicos según la tarea (detección o clasificación) y la función (núcleo o complemento) del *token*:
  - $h_{i,\text{arc}}^{\text{head}}$: Representación como núcleo para **detectar** arcos.
  - $h_{i,\text{arc}}^{\text{dep}}$: Representación como dependiente para **detectar** arcos.
  - $h_{i,\text{label}}^{\text{head}}$: Representación como núcleo para **clasificar** la etiqueta.
  - $h_{i,\text{label}}^{\text{dep}}$: Representación como dependiente para **clasificar** la etiqueta.

Proceso de análisis

Biaffine Attention Mechanism (Dozat y Manning, 2017): método neuronal para asignar pesos a los arcos de dependencias.

- La fórmula *biaffine* (interacción bilineal con transformación afín) valora la compatibilidad sintática entre dos tokens según estos vectores.

$$\text{Score}(i \rightarrow j) = (h_i^{\text{head}})^\top U h_j^{\text{dep}} + W(h_i^{\text{head}} \oplus h_j^{\text{dep}}) + b$$

  - Tranformación bilineal: modela la compatibilidad cruzada entre ambas palabras.
  - Transformación afín: modela la probabilidad de que i sea núcleo y j dependiente.

Ver https://arxiv.org/abs/1611.01734, https://www.emergentmind.com/topics/biaffine-attention-mechanism

Herramientas

- STANZA (modelo *Biaffine Attention*): https://stanfordnlp.github.io/stanza/
- Freeling: https://nlp.lsi.upc.edu/freeling/node/1
- SpaCy: https://spacy.io/](https://spacy.io/
- UD-Pipe: https://ufal.mff.cuni.cz/udpipe
- Apache OpenNLP https://opennlp.apache.org/
- DKPro https://dkpro.github.io/
- Librería Python SuPar: https://pypi.org/project/supar/
- (...)

Actividad final

Analiza con UD-Pipe las siguientes oraciones y localiza los errores.

- `Veinte años hace, poco más o menos, que al declinar una tarde del mes de junio un joven de hermosa presencia atravesaba a caballo los campos pintorescos que riega el Tínima.`
- `Un joven de hermosa presencia atravesaba a caballo los campos pintorescos que riega el Tínima al declinar una tarde del mes de junio hace veinte años poco más o menos.`
- `El sol terrible de la zona tórrida se acercaba a su ocaso entre ondeantes nubes de púrpura y de plata.`

- UD-Pipe https://lindat.mff.cuni.cz/services/udpipe/
- Explicación de las etiquetas: https://universaldependencies.org/u/dep/